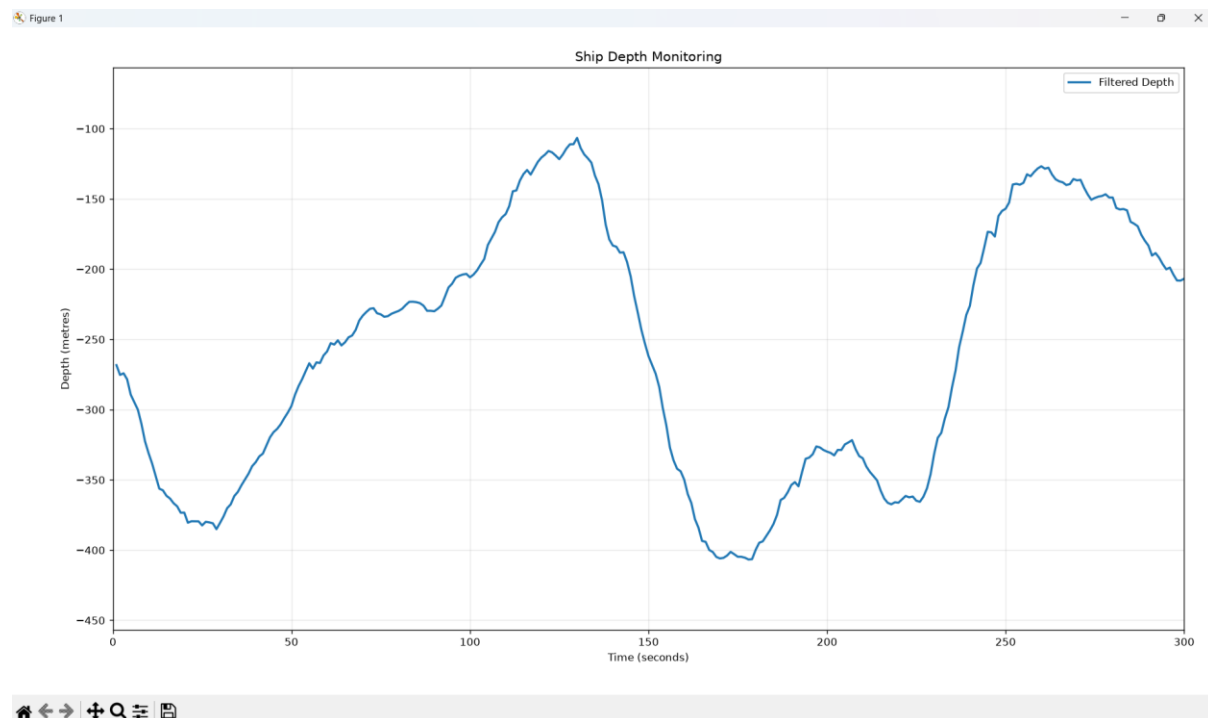


SEDS- ROUND 1 – AVIONICS

Task 1:

Method 1:

1. The CSV file using `pd.read_csv()` function. Depth values are stored in order into a variable named `depth`
2. When there is an error like “#VALUE ” in the data, the `error= “coerce”` command enters the data as 0. The 0 needs to be changed into a reasonable value. Here, I have changed 0 to the average of the values around them.
3. Sometimes the sensor gives spikes due to some internal faults. One such example is when we get a value around -1200. Such readings are also changed to the average of the values around them.
4. The corrected data is stored in `depth_clean`.
5. The corrected data is noisy. It is smoothened using a method called rolling average. Here 5 continuous values are taken and its mean is calculated and saved as `depth_smooth`. Taking the rolling average multiple times smoothenes the data further.
6. Data is plotted using `matlab` library.
7. The `funcanimation` function redraws the `grsaph` every second and executes the update function.



Method 2 (Using Kalman Filter):

The sensor readings jump around because of noise. The Kalman filter maintains its own best estimate. Then that estimate becomes the input for the next measurement.

This is better because, it considers to produce the estimate:

1. What did I think the depth was previously?
2. How much do I trust my sensor?
3. How much do I expect the actual depth to change?

Other features:

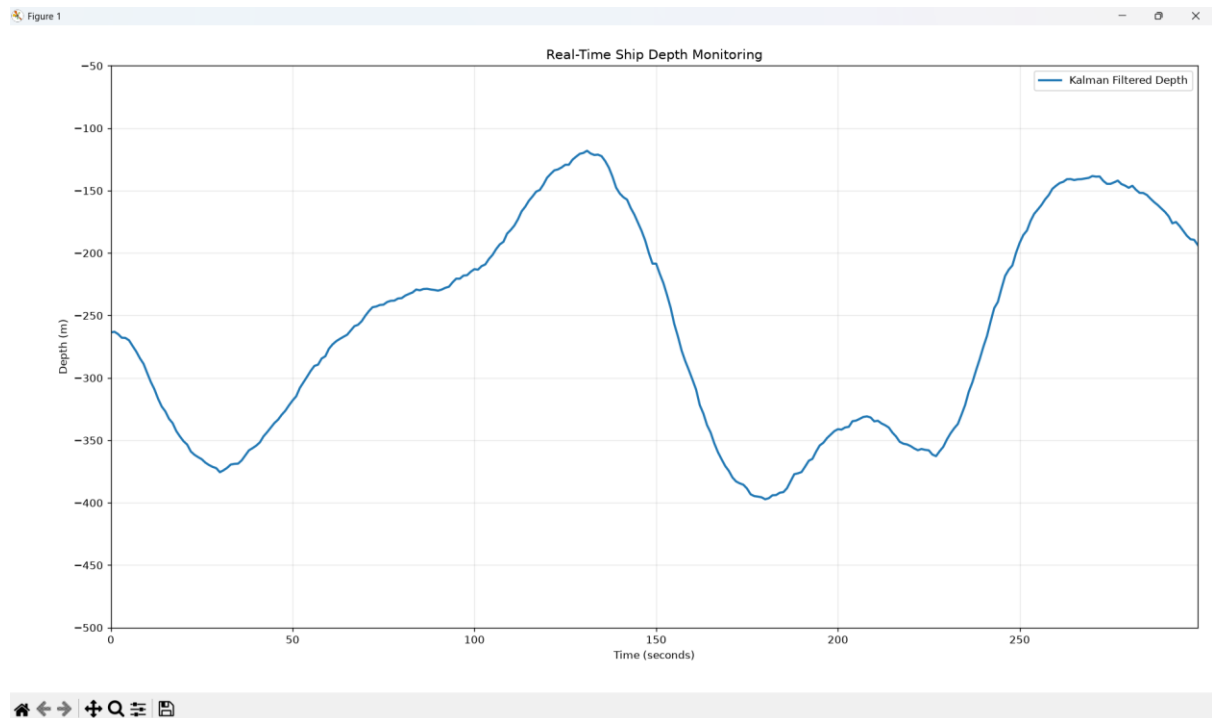
1. When there's an error in the reading. We need not take average. We can temporarily maintain the previous estimate.
2. The sensor data arrives sequentially. The Kalman filter estimates the current depth using the previous estimate and the new measurement, so it doesn't require future data and can operate in real time unlike rolling average which creates a delay.

For live sensor readings, Kalman filter would be a better suit.

Working of Kalman filter:

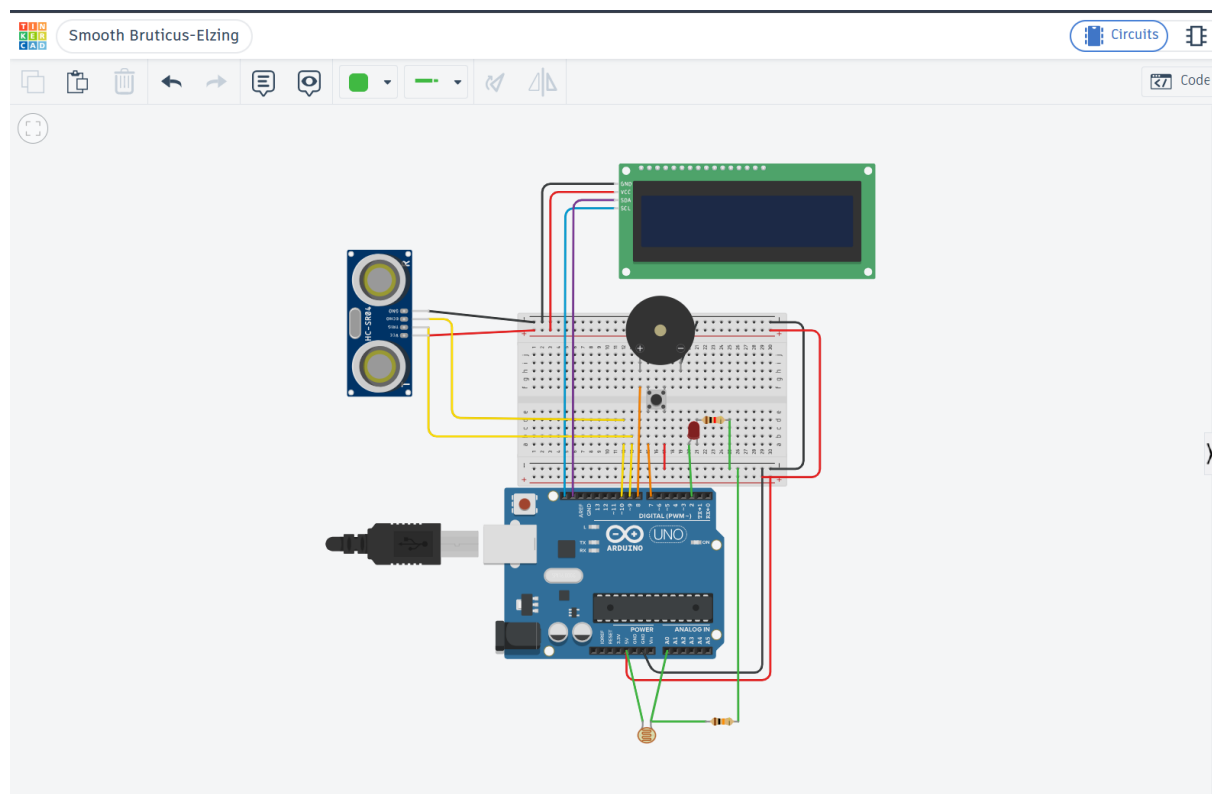
1. It starts with a previous estimate of the depth.
2. A new sensor reading arrives.
3. It calculates how much to trust the new reading versus the previous estimate using the Kalman gain.
4. It combines them to produce a new, smoother estimate.
5. This new estimate becomes the starting point for the next reading.

We can also change how smooth the graph should be by changing the `measurement_noise` variable.



Task 2:

Connections:



Code Explanation:

Functions:

- The getDistance function is defined for reading the distance thru ultrasonic sensor.
- The displayState function changes the LCD screen display to the current state. We make sure it is executed only when the state changes.
- The btn_prs function detects when the button is pressed and returns True.
- The changeState function changes the state to whatever we enter in the bracket. If it was storm or Charybdis, it starts the dangertimer.

1. Initialise all the pins.
2. We use enumeration to define the current state of the ship. It makes the code easier to understand and debug.
3. Initially, the state is set to OPENSEA
4. Sensor readings are taken in. Only if the current state is OPENSEA, the state can change to either STORM or CHARYBDIS.
5. If the STORM or CHARYBDIS state doesn't change for more than 5 sec, the ship goes to WRECKED state.
6. When in STORM or CHARYBDIS, If the button is pressed, the state changes to ANCHOR DROPPED and the ship is saved.
7. If the ship stays in STORM or CHARYBDIS for more than 5 sec, it'll go wrecked and the state would change only after resetting.